- Uses asynchronous programming by default for built-in scalability on cloud platforms

Each of these features was possible in the previous version of ASP.NET but required a fair amount of additional work to set up. With ASP.NET Core, they're all there, ready and waiting to be connected.

Microsoft fully supports ASP.NET Core, so if you want to build a new system, there's no significant reason not to use it. The largest obstacle you're likely to come across is wanting to use programming models that are no longer supported in ASP.NET Core, such as Web Forms or WCF Server, as I'll discuss in the next section.

I hope that this section whetted your appetite to use ASP.NET Core for building new applications. But if you're an existing ASP.NET developer considering whether to convert an existing ASP.NET application to ASP.NET Core, that's another question entirely.

2.4.3 Converting an existing ASP.NET application to ASP.NET Core

By contrast with new applications, an existing application presumably already provides value, so there should always be a tangible benefit to performing what may amount to a significant rewrite in converting from ASP.NET to ASP.NET Core. The advantages of adopting ASP.NET Core are much the same as those for new applications: cross-platform deployment, modular features, and a focus on performance. Whether the benefits are sufficient will depend largely on the particulars of your application, but some characteristics make conversion more difficult:

- Your application uses ASP.NET Web Forms.